Sarcasm Detection with CNN-BiLSTM and CNN-BERT

https://github.com/TEOTD/hybrid-sarcasm-detection

Jawahar Chandrasekaran	Josiah Kiok	Emaan Alfatlawi	Kiran Hegde
DAL719170	JCK210002	EXA200005	KXH230029

Abstract

We attempt to solve the classic problem of sarcasm detection using various multimodal approaches. Sarcasm detection is intrinsically more challenging than other classification problems, as the word order and tone often matters more than the words used themselves. We compare the capabilities of both BiLSTMs and BERT combined with a CNN in order to capture both long and short-range dependencies, also comparing the results to the unary model variants themselves as a baseline. We compare results over two datasets: Reddit Sarcasm (much noisier) and News Headlines (much cleaner). Overall, we find that the CNN seems to slightly improve the BiLSTM model, but does not demonstrate any benefit to the BERT model.

1 Introduction

Sarcasm detection is a useful tool for various NLP tasks, as the mere presence of sarcasm can mitigate the effectiveness of simpler models that cannot easily detect it, like Bag of Words models. Even LLMs have fallen victim to satirical websites and have provided misleading information due to an article humans would immediately identify as satire. In particular, real-time sentiment classification models, especially those receiving data from social media websites like Reddit or Twitter, could easily be misled by the large presence of sarcasm on those and similar sites.

The two datasets which we trained on were "Sarcasm on Reddit", which is a list of 1.3 million labeled comments from Reddit, some of which had the "\s" tag and some of which didn't. The other was much more structured, named "News Headlines Dataset for Sarcasm Detection", which compares headline from *HuffPost*, an actual news site, and *The Onion*, a satire site publishing satirical news with similar formats to other popular news outlets.

2 Task Definition

The task is just simple binary classification. For the Reddit Sarcasm dataset, the input is a user-generated comment (with any \s substrings removed); for the News Headlines dataset, the input is a news headline. In either case, the input is usually around a sentence long. The output label is 1 if sarcastic and 0 if non-sarcastic. We send the input strings through tokenizing and embedding layers as is standard practice.

3 Related Work

Our work roughly follows the outline of "Sarcasm Detection using Hybrid Neural Network", authored by Rishabh Misra and Prahal Arora. There, the authors demonstrate a classification improvement on the same task (84.88% to 89.70%, a 5% improvement) when adding a CNN layer to an LSTM, which we intended to confirm across a range of different datasets. Additionally, we want to also learn if augmenting a transformer with a CNN would also improve the overall model.

As for the News Headlines dataset, we observed the results of the highest rated Kaggle notebook, authored by Madhave Mathur, who achieved Silver on the dataset (there are no publicly available Gold notebooks). It used GloVe/Word2Vec, which had an accuracy of 83.23% on the testing set. Quadeer Shaikh used a Transformer architecture as well, with a much higher accuracy of 93.75%.

4 Data

We evaluate sarcasm detection on two datasets with very different text characteristics. The News Headlines dataset contains short, professionally written headlines with relatively clean grammar and consistent formatting, while the Reddit corpus contains informal, user-generated comments that are much nois-

ier and often require conversational context. Using both lets us compare model behavior under (1) clean, self-contained inputs and (2) realistic social-media style inputs.

4.1 Sarcasm on Reddit

We utilize the Self-Annotated Reddit Corpus (SARC), originally curated by Khodak et al.[1]. This massive dataset comprises approximately 1.3 million comments scraped from Reddit, where ground truth labels are derived from the author's use of the `/s` tag. While the authors note that this tag is generally a reliable indicator of sarcasm, the inverse is not necessarily true; only a fraction of sarcastic comments strictly employ the tag. Although the true distribution of sarcasm in the wild is highly imbalanced (approximately 1:100), we utilize the balanced subset found in `train-balanced-sarcasm.csv` to prevent model bias.

A distinct advantage of this dataset is its inclusion of conversational context. Sarcasm is often dependent on the statement being responded to; a comment that appears literal in isolation may be clearly sarcastic when viewed as a reply. To leverage this, we extract the `parent_comment` field provided in the dataset and concatenate it with the target comment during preprocessing. However, due to the nature of user-generated content on Reddit, the text quality is not particularly high. The dataset is inherently chaotic and models much of the noise—such as slang, irregular formatting, and loose grammar—that characterizes real-world sentiment analysis tasks.

4.2 News Headlines Sarcasm Dataset

To complement the noisy Reddit corpus, we utilize the News Headlines Dataset for Sarcasm Detection, curated by Misra et al. [3, 4]. This dataset mitigates the noise and randomness inherent in social media text by sourcing data from two distinct professional websites: *The Onion* and *HuffPost*. The former is a satire site, where sarcastic examples were collected from the "News in Brief" and "News in Photos" categories; these headlines provide a satirical spin on current events while maintaining the structural form of legitimate news. The latter, *HuffPost*, serves as the source for real, non-sarcastic news headlines.

The primary advantage of this dataset over Twitter-based alternatives is the quality of the text. Because news headlines are written by professionals, they are formally structured and generally free of the spelling errors and informal slang common in tweets. This formal grammar reduces data sparsity and increases the effectiveness of pre-trained word embeddings. Furthermore, unlike tweets which often rely on the context of a thread or a reply to be understood, news headlines are self-contained. This allows our models to isolate the sarcastic elements within the text itself without requiring external context.

Each record in the dataset originally consists of three attributes: `is_sarcastic` (the binary label), `headline` (the text content), and `article_link`. For our experiments, we extract the headlines and labels, normalizing the format to match our pipeline. While there is a potential concern that models might overfit to the specific writing styles of the two publications rather than learning general sarcasm, the use of word embeddings helps mitigate this by focusing on semantic relationships rather than purely stylistic markers.

4.3 Shared Preprocessing

To ensure reproducibility and rigorous evaluation, we implemented a standardized preprocessing pipeline for both datasets. A global random seed of 42 was initialized for all non-deterministic operations.

Data Splitting

We employed a stratified splitting strategy to preserve the original class distribution (the ratio of sarcastic to non-sarcastic samples) across all subsets. The dataset was partitioned as follows:

- **Test Set:** 20% of the total data was held out for final evaluation.
- **Validation Set:** 10% of the remaining training data was used for hyperparameter tuning and model checkpointing.
- **Training Set:** The remaining portion was used for model training.

Sanitization

Prior to splitting, we performed data hygiene steps:

- **HTML Removal:** All text inputs were cleaned using `BeautifulSoup` to remove HTML tags and artifacts.
- **Deduplication:** Exact duplicate entries were removed to avoid data leakage across train, validation, and test sets.
- **Whitespace Normalization:** Irregular spacing and newline characters were collapsed into single spaces.

5 Methodology

5.1 Overall approach

The main models created were the hybrid CNN-BiLSTM baseline and CNN-Bert, both of which pooled results from their respective models into a final feedforward layer before classification. We also create non-hybrid versions of these models (that is, CNN, a BiLSTM, and BERT) to verify individual correctness. We test all five of the models on final datasets.

At the end, we run XAI in order to verify that the model is learning as expected.

5.2 Model 1: CNN-BiLSTM baseline

Our first model is a hybrid CNN-BiLSTM network for sarcasm classification. It is a simplified version of the architecture from Misra and Arora [2]. A CNN captures local n-gram patterns in the headline and a bidirectional LSTM runs in parallel to model longer-range context across the sentence. We implement this architecture in PyTorch and use it as our main non-transformer baseline.

Input Representation

While the general data cleaning and splitting is described in Section 4, this specific architecture requires discrete token inputs. We apply the following preprocessing steps specific to the CNN-BiLSTM:

- **Tokenization:** We lowercase each headline and tokenize it using simple whitespace splitting.
- **Vocabulary Building:** We build a vocabulary on the training split only, keeping all tokens with a frequency of at least 2. Rare tokens are mapped to a special `<unk>` token.
- **Padding:** We add a special `<pad>` token to handle variable lengths. Each headline is truncated or padded to a fixed length of 40 tokens.

This results in input sequences of length 40 over a vocabulary of approximately 13,710 tokens. The embedding layer is randomly initialized and trained jointly with the model.

The preprocessing for the CNN-BiLSTM model is intentionally simple:

- we lowercase each headline and tokenize it using whitespace splitting
- we build a vocabulary on the training split only, keeping all tokens with frequency at least 2 and the rare tokens are mapped to an `<unk>` token
- we add two special tokens `<pad>` and `<unk>` to handle padding and unknown words
- each headline is truncated or padded to a fixed length of 40 tokens, using `<pad>` for shorter headlines

This gives us input sequences of length 40 over a vocabulary of 13 710 tokens (including special tokens). For fair comparison, we reuse the same train/validation/test splits for the other models.

Architecture

Given a padded sequence of token ids $(w_1, \ldots, w_{40})$, we look up a learned embedding for each token. The embedding layer maps each token to a 100-dimensional vector, so a headline is represented as a matrix $\mathbf{E} \in \mathbb{R}^{40\times 100}$. The embedding matrix is randomly initialized and trained jointly with the rest of the model.

CNN branch. To capture local n-gram patterns, we apply several 1D convolutions over the sequence of embeddings. We use filter widths of 3, 4, and 5 tokens with 100 filters for each width. For each filter we apply a ReLU non-linearity followed by max-over-time pooling across the time dimension. This gives us a single scalar per filter, essentially a 100-dimensional vector per filter width. We then concatenate the outputs from all three filter sizes to obtain a CNN feature vector of dimension $100 \times 3 = 300$.

BiLSTM branch. In parallel, we process the same sequence of embeddings with a one-layer bidirectional LSTM. The hidden size is 128 in each direction. We use the final forward and backward hidden states from the last LSTM layer and concatenate them to obtain a 256-dimensional BiLSTM feature vector. This vector summarizes the overall meaning of the headline, including information that may only appear near the end of the sentence.

Classifier. The CNN and BiLSTM feature vectors are concatenated, giving a combined feature vector of size $300 + 256 = 556$. We apply dropout with rate 0.5 and then pass the result through a fully connected layer with a single output unit. This outputs a real valued logit which is converted to a probability with a sigmoid at evaluation time. We use the binary cross-entropy with logits loss (`BCEWithLogitsLoss` in PyTorch) for training.

Component	Setting
Embedding dimension	100
BiLSTM hidden size	128 (per direction)
Convolution filter sizes	3, 4, 5 tokens
Filters per size	100
Max sequence length	40 tokens
Batch size	64
Optimizer	Adam, learning rate 10^{-3}
Dropout rate	0.5
Training epochs	5

Table 1: Hyperparameters used for the CNN-BiLSTM baseline.

Training setup

We train the CNN-BiLSTM model only on the training split of the News Headlines dataset. The main hyperparameters we use are summarized in Table 1.

We fix the random seed in NumPy and PyTorch for reproducibility. During training we monitor accuracy on the validation split at the end of each epoch. Whenever the validation accuracy improves, we save the current model parameters and the vocabulary to a checkpoint file. At the end of training we reload the best checkpoint based on validation accuracy and evaluate it on the held-out test set.

5.3 Model 2: CNN-BERT

Our second hybrid model combines a pre-trained transformer encoder (BERT) with a CNN feature extractor. The goal is to test whether explicitly capturing local n-gram patterns on top of contextual BERT embeddings provides any benefit, analogous to the CNN branch used in the CNN-BiLSTM baseline. We implement this model in PyTorch using the Hugging Face Transformers library for the BERT backbone.

Input Representation

As described in Section 4, we use the same train/validation/test splits across all models. For CNN-BERT, we do not build a custom vocabulary. Instead, each input is tokenized using BERT's WordPiece tokenizer. We append `[SEP]`, and we construct an attention mask to distinguish real tokens from padding. Inputs are truncated or padded to a fixed maximum length of 40 tokens to match the sequence length used in our other experiments.

For Reddit examples, we incorporate conversational context by concatenating the `parent_comment` and the target comment into a single sequence separated by `[SEP]`. This allows the transformer to jointly encode both the context and the reply in one forward pass.

Architecture

Let the tokenized sequence length be $L = 40$. Passing the sequence through BERT yields contextual token embeddings:

$$\mathbf{H} = (\mathbf{h}_1, \ldots, \mathbf{h}_L), \quad \mathbf{h}_i \in \mathbb{R}^d,$$

where d is the hidden size of the BERT model. We then compute two representations in parallel and combine them for classification.

CNN feature over BERT embeddings. To capture local phrase-level cues, we apply 1D convolutions across the token dimension of $\mathbf{H}$. We use filter widths of 3, 4, and 5 tokens with 100 filters per width. Each convolution is followed by a ReLU non-linearity and max-over-time pooling, producing a fixed-size vector for each filter width. Concatenating these pooled vectors yields a 300-dimensional CNN feature vector.

Classifier. We concatenate the BERT pooled feature and the CNN feature vector and apply dropout before a final linear layer that outputs a single logit. Training uses binary cross-entropy with logits (`BCEWithLogitsLoss`). The full model is fine-tuned end-to-end so that gradients update both the CNN/classifier layers and the BERT parameters.

Training setup

CNN-BERT differs from CNN-BiLSTM primarily in optimization and input processing. Since BERT is pre-trained and sensitive to learning rate, we use AdamW with a smaller learning rate (2×10^{-5}) and weight decay. We train for up to 5 epochs with early stopping based on validation accuracy. Because transformer fine-tuning is more computationally expensive, we use the same effective batch size when possible and apply gradient clipping for stability.

This architecture directly tests whether an explicit CNN layer provides complementary local features beyond what is already captured by BERT's self-attention, and it also enables comparison of attribution patterns between BERT and CNN-BERT in our XAI analysis.

5.4 XAI and analysis setup

To identify which words influence sarcasm detection, we use a gradient-based saliency method. We calculate the gradient of the model's output logit with

respect to the input embeddings. The magnitude of the gradient reveals how strongly a specific token affects the final prediction.

Specifically, for each token w_i in a sentence, we compute importance as:

$$\text{importance}(w_i) = \left\| \frac{\partial f(\mathbf{x})}{\partial \mathbf{e}_i} \right\|_2 \tag{1}$$

where $f(\mathbf{x})$ is the model's output logit and $\mathbf{e}_i$ is the embedding vector for token w_i. High values indicate the token played a major role in the model's decision.

We visualize these scores by highlighting words with varying color intensities, allowing us to see exactly which terms the model links to sarcasm. We compare BERT and CNN-BERT to see if adding CNN features changes which words the model relies on.

We analyze both correct and incorrect predictions to understand:

- which tokens signal sarcasm most strongly
- whether models use obvious markers (like exaggeration) or subtle context
- which words mislead the model in failed cases

6 Experiments and Analysis

6.1 Experimental setup

Data splits

For both datasets, we use an 80/10/10 split strategy. Specifically, we first reserve 20% of the data as a held-out test set, then from the remaining 80%, we allocate 10% for validation (which is 12.5% of the remaining data). This gives us approximately:

- 70% training data
- 10% validation data
- 20% test data

The News Headlines dataset contains 28,619 headlines; the SARC Reddit dataset contains 1.3 million comments but we use a subset for computational efficiency. All splits are created with a fixed random seed (42) to ensure reproducibility. We preserve the same splits across all models to enable fair comparison.

Evaluation Metrics

We assess model performance on the test set using standard metrics:

- **Accuracy**: percentage of total correct predictions
- **Precision**: proportion of predicted sarcastic items that are actually sarcastic
- **Recall**: proportion of actual sarcastic items correctly identified
- **F1-Score**: harmonic mean of precision and recall
- **ROC-AUC**: measures the ability to distinguish between classes across all decision thresholds

During training, we track validation accuracy and save checkpoints whenever performance improves. We report final test results using the checkpoint with the highest validation accuracy.

Model Comparison

To ensure a fair comparison, we standardized the training environment:

- **Data**: Identical train/validation/test splits for all models
- **Determinism**: Fixed random seed (42) for NumPy, PyTorch, and Python
- **Parameters**: Consistent batch size (64) and sequence length (40 tokens)
- **Stopping**: Early stopping if validation accuracy does not improve for 3 epochs

The primary differences are architectural and hyperparameter-based. CNN-BiLSTM uses 100-dimensional learned embeddings with a learning rate of 10^{-3}. In contrast, BERT models use 512-dimensional contextual embeddings and a lower learning rate of 2×10^{-5}.

Implementation Details

We implemented all models in PyTorch 2.9.1. For BERT components, we used the Hugging Face Transformers library (v4.57.1). Training runs on CUDA-enabled GPUs where available.

A single script (`run.py`) manages the entire pipeline, handling:

- automated data download via Kaggle Hub

- preprocessing and vocabulary creation
- model training and checkpointing
- evaluation and XAI visualization

Models typically converge within 5–10 epochs. The CNN-BiLSTM trains in roughly 2–3 minutes per epoch, while the more complex CNN-BERT requires 5–7 minutes per epoch.

6.2 Results on News Headlines

Table 2 shows the performance of the models on the News Headlines dataset. We report the best validation accuracy over 5 epochs and the test accuracy of the corresponding checkpoint.

Model	Test Accuracy
CNN-BiLSTM	0.8427
CNN-BERT	0.8406
CNN	0.8354
LSTM	0.8326
BERT	0.8523

Table 2: Performance of the various models on the News Headlines sarcasm dataset.

The CNN-BiLSTM model reaches around 83% accuracy on the validation set and about 84% accuracy on the test set. Training accuracy is higher at above 96% by the last epoch), which suggests some amount of overfitting but the gap is not extreme. Overall, this hybrid CNN-BiLSTM architecture already gives a strong baseline on headlines and it provides a good point of comparison for later models and analysis.

The results of CNN-BERT leave much to be desired, as BERT on its own beats the hybrid models. We speculate this is partially due to model training and evaluation variance, but the results seem to indicate that many of the pieces of information captured by CNN were already handled by BERT.

Results on Reddit Dataset

Table 3 shows the performance of the models on the News Headlines dataset. We report the best validation accuracy over 5 epochs and the test accuracy of the corresponding checkpoint.
On the noisy Reddit dataset, both hybrid models seem to drastically underperform, though again, the minor changes in results can be attributed to variance. The best-performing model on the test set is the LSTM, with an accuracy of 69.73%. In particular,

Model	Test Accuracy
CNN-BiLSTM	0.6962
CNN-BERT	0.6823
CNN	0.6797
LSTM	0.6973
BERT	0.6632

Table 3: Performance of the various models on the Reddit sarcasm dataset.

BERT does seems to handle the noise much worse, being consistently worse than the LSTM rather than better, unlike the previous dataset.

It should be noted that the final hybrid CNN-BERT model does better than either CNN or BERT individually.

6.3 Error Analysis and XAI

To understand model behavior, we applied gradient-based explainability analysis to a sample of test predictions. We examined both correct and incorrect classifications across sarcastic and non-sarcastic categories.

Correctly Classified Examples

When models correctly identify sarcasm, they typically focus on words that signal contradiction or exaggeration. For example, in the headline:

> *"Nation's economists announce they've given up"*

The CNN-BiLSTM model weighted the phrase "given up" heavily, noting its contrast with the expected professional tone. CNN-BERT highlighted both "economists" and "given up," capturing the irony.

For non-sarcastic headlines, models focused on neutral, descriptive terms. in:

> *"New study reveals link between sleep and health"*

Both models emphasized standard reporting words like "study," "reveals," and "link."

Visually, CNN-BiLSTM spreads importance evenly due to its sequential processing. In contrast, CNN-BERT targets specific words more sharply, reflecting the combination of CNN filters and BERT's contextual attention.

Failure Cases

Models struggle with sarcasm that requires external knowledge. Consider:

> *"Congress works over holiday weekend to pass critical legislation"*

Without knowing about legislative gridlock, the text appears neutral. Both models classified this as non-sarcastic, focusing on generic tokens like "Congress" rather than the implied irony.

Models also fail when absurd scenarios use standard news language:

> *"Local man confident in ability to fix sink despite zero plumbing experience"*

While the contrast between "confident" and "zero experience" is obvious to humans, the sentence structure mimics a standard human-interest story, leading the models to miss the sarcasm.

Challenging Sarcasm Types

Our analysis identifies four categories that remain difficult for these models:

- **Context-dependent**: Sarcasm relying on real-world facts (politics, culture) rather than just word choice.
- **Subtle exaggeration**: Cases where the absurdity is slight or normalized within the dataset.
- **Deadpan delivery**: Headlines with a neutral tone and no obvious markers like exclamation points or emotional words.
- **Domain-specific**: Sarcasm regarding niche topics (e.g., academia, tech) where training data is scarce.

Model Comparison via XAI

Comparing the models reveals distinct behaviors. CNN-BERT is generally more confident in its predictions due to its pre-trained embeddings. However, this is a double-edged sword: when CNN-BERT is wrong, it is often confidently wrong.

The CNN component helps capture local phrases (n-grams) in both architectures. However, this benefit is larger for the BiLSTM baseline. Since BERT's self-attention already captures complex dependencies, adding CNN layers to BERT yields diminishing returns.

In summary, while the models learn effective heuristics, focusing on incongruity and tone, they lack the world knowledge required for complex sarcasm detection.

7 Conclusion and Future Work

In this project, we studied sarcasm detection as a binary classification task across two complementary domains: clean, professionally written news headlines and noisy, user-generated Reddit comments. We compared two hybrid architectures (CNN-BiLSTM and CNN-BERT) against their unary counterparts (CNN, LSTM/BiLSTM, and BERT) to test the common hypothesis that a CNN layer can strengthen sequence models by capturing salient local n-gram cues.

Summary of results. On the News Headlines dataset (Table 2), BERT achieves the strongest overall performance (0.8523 test accuracy). The CNN-BiLSTM hybrid performs competitively (0.8427) and slightly outperforms the unary CNN and LSTM baselines, which suggests that combining local phrase features (CNN) with longer-range context modeling (BiLSTM) is beneficial on clean, short text. However, CNN-BERT does not improve over BERT; in fact, it slightly underperforms (0.8406). This indicates that BERT's contextual representations already capture much of the local dependency signal that the CNN branch is intended to provide, making the additional convolutional features largely redundant on this dataset.

On the Reddit dataset (Table 3), performance drops across all models due to informal language, higher variance in writing style, and stronger reliance on conversational context. Interestingly, the LSTM baseline attains the highest test accuracy (0.6973), with CNN-BiLSTM close behind (0.6962). In contrast, BERT performs worse (0.6632), suggesting that off-the-shelf transformer fine-tuning is less robust under this domain shift without additional adaptation. While the CNN-BERT hybrid is not the strongest model overall on Reddit (0.6823), it does outperform both unary CNN (0.6797) and unary BERT (0.6632), implying that explicit local feature extraction can help stabilize transformer-based models in noisier settings even when it does not help in cleaner domains.

Insights from XAI. Gradient-based saliency analysis broadly aligned with our quantitative findings. For correctly classified sarcastic examples, models tended to assign high importance to incongruity markers and locally meaningful phrases (e.g., exaggerated claims or contradictory wording). Failure

cases frequently involved sarcasm that depends on external knowledge, cultural context, or subtle deadpan delivery, where the surface form resembles non-sarcastic text. Qualitatively, CNN-BiLSTM often distributed importance more broadly across the sequence, while BERT-based models tended to concentrate attribution on fewer tokens, consistent with their contextual encoding behavior.

Limitations. Our evaluation has several limitations. First, the Reddit labels depend on the `/s` tag, which introduces label noise and does not capture all sarcastic cases. Second, both datasets can contain stylistic artifacts that models may exploit (e.g., publication-specific headline patterns). Third, computational constraints required subsetting and limited hyperparameter search, and results may vary across random seeds and fine-tuning settings. Finally, our maximum sequence length was fixed for consistency, which may truncate useful Reddit context.

Future work. Several directions would strengthen both performance and interpretability:

- **Better context modeling on Reddit:** represent `parent_comment` and reply as two segments (with explicit separators) and explore hierarchical or conversational encoders that better reflect thread structure.
- **Domain adaptation for transformers:** continue pre-training on unlabeled Reddit text (DAPT) or use adapters/LoRA to improve robustness to slang, informal grammar, and domain shift.
- **Stronger transformer backbones:** evaluate RoBERTa/DeBERTa-style encoders and compare whether CNN augmentation helps when the base encoder changes.
- **More rigorous evaluation:** run multiple seeds and report mean/variance, and analyze calibration (since BERT-style models can be confidently wrong).
- **Richer attribution methods:** complement saliency with Integrated Gradients or perturbation-based approaches to verify stability of explanations across methods.

Overall, our experiments suggest that CNN augmentation provides a clearer benefit when paired with LSTM-style sequence modeling, but yields limited (or negative) returns on top of BERT for clean headlines. On noisier Reddit text, improving domain robustness and context utilization appears more impactful than adding additional local feature extractors alone.

References

[1] Mikhail Khodak, Nikunj Saunshi, and Kiran Vodrahalli. A large self-annotated corpus for sarcasm. In *Proceedings of the LREC 2018*, 2018. Dataset available at https://www.kaggle.com/datasets/danofer/sarcasm/data.

[2] Rishabh Misra and Prahal Arora. Sarcasm detection using hybrid neural network. *arXiv preprint arXiv:1908.07414*, 2019.

[3] Rishabh Misra and Prahal Arora. Sarcasm detection using news headlines dataset. *AI Open*, 4:13–18, 2023.

[4] Rishabh Misra and Jigyasa Grover. *Sculpting Data for ML: The first act of Machine Learning*. 01 2021.